AFSMS — Comprehensive Walkthrough Guide

Automated Faculty Student Management System

Project P9-T4 · Sugubete, Maximilian, Mitrache

What Is This System?

AFSMS (Automated Faculty Student Management System) is a full-stack web application built for the **Faculty of Automation, Computers and Electronics at the University of Craiova**. It manages the complete academic lifecycle: student records, curricula, grades, document workflows, audit trails, backups, and notifications — all in one place, behind a role-based access control system that mirrors the real university structure.

The system has two halves: a **React + Vite frontend** deployed as a GitHub Pages SPA, and a **Node.js / Express backend** connected to a **PostgreSQL** database.

---

1 Part 1 — The Database Layer (Bottom)

Everything starts here. The schema is in `src/schema.sql` and defines 8 logical modules.

Module 1 — Security & Users

The foundation of all access control.

- `ROLE` — defines the four roles in the system: `STUDENT`, `PROFESSOR`, `SECRETARIAT`, `ADMIN`. Each role has a unique `code` used throughout the backend for permission checks.
- `PERMISSION / OPERATION` — fine-grained action definitions (e.g., `READ_GRADES`, `EXPORT_REPORT`) that can be linked to roles via `ROLE_PERMISSION` and `ROLE_OPERATION` junction tables.
- `USER_ACCOUNT` — every person in the system has a record here. The critical field is `sso_subject`, a unique identifier that comes from the university's SSO (Single Sign-On) system. This is what ties a university identity to an AFSMS account. The field `account_status` can be `ACTIVE` or `SUSPENDED`.
- `USER_ROLE` — maps users to roles. A user has exactly one active role at a time.
- `AUTH_SESSION` — stores session tokens with expiry and IP address for traceability.
- `USER_GROUP` / `USER_GROUP_MEMBER` — a simple grouping mechanism used for mass email notifications. A group like "Year 1 CTI" contains user accounts.
- `OUTLOOK_NOTIFICATION` — records of every bulk email sent through the system, including recipient list, subject, body preview, and delivery status.

Module 2 — Academic Data

The core of what makes the system academic.

- `ACADEMIC_YEAR` — records like `2025/2026`. Only one can be `is_active = TRUE` at a time. Used as a reference point for curricula and reports.
- `SPECIALIZATION` — a program of study (e.g., "Calculatoare și Tehnologia Informației", "Automatică și Informatică Aplicată"). Has a `degree_level` field (`LICENTA`, `MASTER`).
- `STUDY_FORMATION` — a specific group of students, e.g., "CTI Year 2 Group A". Has `study_year`, `education_form` (ZI/ID/IFR), and `group_index`.
- `STUDENT` — the student record, linked to a study formation. Contains `registration_number` (e.g. `MAT1234`), names, email, enrollment date, and status (`ENROLLED`, `ACTIVE`, `GRADUATED`, `WITHDRAWN`).
- `CURRICULUM` — a plan of study for a specialization, with a `valid_from` date. Can be snapshotted.
- `CURRICULUM_SNAPSHOT` — an immutable copy of a curriculum at a point in time. Used to preserve the rules a student was enrolled under, even after the curriculum changes.
- `DISCIPLINE` — a course/subject belonging to a curriculum. Has `semester`, `ects_credits`, and `evaluation_type` (EXAM, COLLOCVIU, VP).
- `STUDENT_CURRICULUM` — the enrollment record: which student is in which curriculum, under which study formation.
- `GRADE` — the result. Linked to a student, a discipline, and optionally a curriculum snapshot. Fields: `value` (1–10, 0 = absent), `exam_session`, `grading_date`, and `graded_by_user_id` (the professor who entered it).
- `DATA_IMPORT_JOB` — records of bulk Excel/CSV imports, so you know when a batch of grades was uploaded and by whom.

Module 3 — Documents

`DOCUMENT` — a file or record that goes through an approval workflow. Has `type`, `status` (`DRAFT`, `PENDING`, `APPROVED`, `REJECTED`, `MODIFICATION`), `author_id`, `assigned_to_user_id`, `file_path`, and `content` (extracted text for full-text search). Supports physical file storage on disk.

Module 4 — Reports

- `REPORT_DEFINITION` — a named, reusable report template (e.g., "Electronic Grade Centralizer"). Has visibility (`PUBLIC` / `PRIVATE`).
- `REPORT_REQUEST` — an instance of running a report, with filters stored as JSON.
- `E_GRADE_CENTRALIZER` — the specific data model for the grade centralizer report.
- `REPORT_ROW` — individual rows of a generated report.
- `REPORT_EXPORT` — tracks every time a report was exported (PDF, CSV, XLSX, XML), by whom, and where the file was stored.

Module 5 — Audit & Recovery

The most architecturally interesting module.

- `AUDIT_LOG_ENTRY` — the central ledger. Every create, update, and delete in the system writes a record here with: `actor_user_id`, `action_type` (INSERT, UPDATE, DELETE, ROLLBACK, PITR), `entity_type` (the table name), `entity_id`, `before_snapshot_json`, and `after_snapshot_js` This is what makes rollback possible.
- `AUDIT_LOG_ARCHIVE` — a mirror table. Logs older than 5 years are automatically moved here (not deleted), satisfying the institutional 5-year retention policy.
- `BACKUP_JOB` — records of every backup run: type (AUTOMATIC, MANUAL), start/end times, storage location (a JSON file path), and status.
- `BACKUP_CONFIG` — a single-row table controlling the backup scheduler: `enabled` (boolean), `cron_expression` (e.g., `0 2 * * *` for 2 AM daily).
- `RECOVERY_POINT` / `ROLLBACK_OPERATION` — structural tables for tracking recovery operations.

Module 6 — System Configuration

`SYSTEM_SETTING` — a key-value store for institutional metadata: faculty name, institution name, dean's name, system email, portal notices. Editable by Admin from the UI.

Performance Indexes

The schema defines indexes on the most-queried paths: `STUDENT(registration_number)`, `STUDENT(study_fo` `GRADE(student_id)`, `DOCUMENT(author_id, type, created_at)`, `AUDIT_LOG_ENTRY(entity_type, entity_id)`, `AUDIT_LOG_ENTRY(actor_user_id)`, `AUDIT_LOG_ENTRY(occurred_at)`.

2 Part 2 — The Backend

The backend is a **Node.js / Express 5** application (`backend/src/server.js`). It listens on port 8080 (configurable via `PORT` env variable) and exposes a REST API under `/api/`.

Server Bootstrap

On startup, the server does three things before taking requests:

1. **Initializes the scheduler** (`schedulerService.initScheduler()`), which reads `BACKUP_CONFIG` from the database and either starts a cron job for automatic backups or skips it. It also always schedules the audit log archiving job (runs daily at 1 AM).
2. **Applies security headers** to every response: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, a minimal CSP, and removes `X-Powered-By`.

3. **Initializes the `SYSTEM_SETTING` table** with default values (faculty name, dean name, etc.) on first run, using `ON CONFLICT DO NOTHING` so it never overwrites saved values.

CORS is controlled by the `ALLOWED_ORIGINS` environment variable (comma-separated list). The frontend origin (`http://localhost:5173` for dev and the GitHub Pages URL for production) is always allowed.

A UTF-8 middleware intercepts every response and ensures `application/json` and `application/xml` content types always include `; charset=utf-8`, which is essential for Romanian diacritics (ș, ț, ă, â, î).

Database Layer (`src/db.js`)

Wraps `pg.Pool` from the `pg` package. Key exports: `db.query(sql, params)` for simple queries and `db.transaction(async (client) => {...})` for atomic multi-step operations. The transaction helper manages `BEGIN / COMMIT / ROLLBACK` automatically.

Connection string is assembled from `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` environment variables.

Middleware

`authMiddleware.js` exports two functions:

`requireAuth` — extracts the `Authorization: Bearer <token>` header, verifies the JWT against `JWT_SECRET`, and attaches `{ userId, role, fullName }` to `req.user`. Returns 401 if missing, 403 if invalid or expired.

`requireRole(allowedRoles)` — a factory that returns middleware checking whether `req.user.role` is in the provided array. Returns 403 if not. Used on every route that has access restrictions.

`errorHandler.js` — the global error handler (last middleware). Formats all errors into a consistent `{ error: true, message, code }` JSON structure. Recognizes PostgreSQL error codes (e.g., `23505` for unique violations, `23503` for FK violations) and maps them to user-friendly messages with resolution hints. Also recognizes custom error properties (`err.customMessage`, `err.customCode`) set by the audit service.

Routes Overview

All routes are prefixed `/api/`. Here is what each file owns:

- **`/api/mock-sso`** — simulates the university's Identity Provider. Accepts a login request with `email`, `role`, and `full_name`, and issues a signed JWT (`SSO_MOCK_SECRET`) that the frontend can then exchange for a real AFSMS session token. This exists only because the real university SSO is not publicly accessible.
- **`/api/auth`** — handles two flows:
  - `POST /exchange-token` — the main login flow. Receives the SSO token, verifies it, finds or creates the user in `USER_ACCOUNT`, syncs the role, auto-creates the `STUDENT` record if the role is `STUDENT`, and issues an AFSMS JWT (`JWT_SECRET`, 8-hour expiry).
  - `POST /register` — a direct registration path (no SSO), assigns `STUDENT` role by default.

- `/api/academic` — the largest route file (∼49 KB). Covers:
  - `GET /dashboard/stats` — role-aware statistics for the dashboard card grid. Each role sees different numbers.
  - Student CRUD: list, add, update, delete, import from Excel/CSV.
  - Curriculum CRUD: list, add, update, delete, snapshot.
  - Discipline CRUD: list, add, update, delete.
  - Study Formation CRUD.
  - Grade entry: add individual grade, bulk Excel import, fetch grades per student or discipline.
- `/api/admin` — ADMIN-only (except user listing which is also accessible to SECRETARIAT and PROFESSOR for the document forward workflow):
  - Role management (create, update, delete roles).
  - User management (create, update, delete users, assign roles, suspend accounts).
  - Backup management (trigger manual backup, list backup history, download backups, update backup schedule, restore from a JSON backup file).
- `/api/audit` — the audit and recovery endpoints:
  - `GET /` — paginated list of audit log entries, joined with user names.
  - `POST /rollback/:logId` — reverses a single audit log entry (undo an INSERT by deleting, undo an UPDATE by restoring `before_snapshot_json`, undo a DELETE by re-inserting).
  - `POST /pitr` — Point-in-Time Recovery. Takes a `targetTimestamp` and replays all audit log entries after that timestamp in reverse order, effectively winding the database back.
- `/api/documents` — document lifecycle:
  - `GET /` — advanced search with filters (type, author, date range, content keyword, author keyword).
  - `POST /` — upload a new document with file. Extracts text from PDF, DOCX, XLSX, TXT, CSV, XML for full-text search.
  - `PUT /:id/status` — update document status (approve, reject, request modification).
  - `PUT /:id/forward` — forward document to a specific user.
  - `PUT /:id/reupload` — replace the file on a document in MODIFICATION status.
  - `GET /download/:id` — stream the file to the browser.
  - `DELETE /:id` — soft-delete the record and remove the physical file.
- `/api/reports` — the centralizer and export engine:
  - `GET /centralizer` — runs the master query that joins students, formations, disciplines, and grades through a dynamic filter set (specialization, study year, education form, semester, discipline, group index, academic year, degree level, and free-text search). Returns the raw data.
  - `GET /centralizer/export` — same query, but exports to CSV, XLSX, or XML depending on `format` query param.
  - `GET /transcript/:studentId` — generates a full academic transcript for a student.
- `/api/notifications` — mass email:

  - `POST /send` — resolves recipients (either a `USER_GROUP` or a `STUDY_FORMATION`), calls the configured email service, and logs the send in `OUTLOOK_NOTIFICATION`.

- `/api/groups` — user group management (CRUD for groups and members).
- `/api/lookup` — read-only reference data endpoint. Returns formations, specializations, academic years, and curricula in lightweight format for populating dropdowns in the UI.
- `/api/public` — unauthenticated endpoint exposing a portal notice and the faculty/institution name from `SYSTEM_SETTING`, used by the public-facing landing page.
- `/api/config` — ADMIN-only system configuration:
  - System settings (key-value pairs).
  - Academic year management.
  - Specialization management.
  - Backup schedule configuration (enable/disable, set cron expression).

Services

`auditService.js` — the backbone of data integrity. Provides:

- `logAudit(client, actorUserId, actionType, moduleName, entityType, entityId, before, after)` — the raw insert function. Always called inside a transaction using the same `client` so the audit entry and the data change are atomic.
- `auditableUpdate(actorUserId, moduleName, tableName, id, updateFields)` — fetches the before snapshot, performs the UPDATE, then logs.
- `auditableInsert(actorUserId, moduleName, tableName, insertFields)` — performs the INSERT, then logs with `null` before snapshot.
- `auditableDelete(actorUserId, moduleName, tableName, id)` — fetches before snapshot, performs the DELETE, then logs with `null` after snapshot.
- `archiveOldLogs()` — moves entries older than 5 years from `AUDIT_LOG_ENTRY` to `AUDIT_LOG_ARCHIVE`.

Every data-modifying route in the system that goes through one of these functions automatically gets a full audit trail with before/after snapshots.

`backupService.js` — iterates through the `TABLES_TO_BACKUP` list (16 tables), dumps all rows from each as JSON, writes a timestamped file to `backend/backups/`, and records the backup job in `BACKUP_JOB`.

`schedulerService.js` — uses `node-cron` to manage two scheduled jobs:

- Automatic backup — reads `BACKUP_CONFIG` on startup and starts a cron job if enabled.
- Audit archiving — always scheduled, runs daily at 1 AM.

Provides `updateSchedule(cronExpression, enabled)` called by the admin backup config route.

`emailService.js` — a router that selects the right implementation based on environment variables: Graph API (`graphEmailService.js`) for Microsoft 365 integration, SMTP (`smtpEmailService.js`)

for standard mail servers, or Mock (`mockEmailService.js`) for development (just logs to console). The `sendBulk({ from, recipients, subject, text, html })` interface is the same regardless of implementation.

`importService.js` — parses Excel/CSV grade sheets uploaded by professors and bulk-inserts grades with proper validation and audit logging.

3 Part 3 — Authentication Flow

Understanding how a user logs in is key to understanding the whole system.

1. The user lands on the AFSMS login page and clicks "Login via University SSO".
2. The frontend calls `POST /api/mock-sso/login` with email and role. The mock SSO returns a signed JWT (`ssoToken`).
3. The frontend immediately calls `POST /api/auth/exchange-token` with that `ssoToken`.
4. The backend verifies the SSO token signature, looks up or creates the user in `USER_ACCOUNT`, syncs the role from the SSO claims, auto-creates a `STUDENT` record if needed, and issues a new short-lived AFSMS JWT (8 hours).
5. The frontend stores this AFSMS token in the `AuthContext` and sends it as `Authorization: Bearer <token>` on every subsequent API call.
6. `requireAuth` verifies every protected request. `requireRole` enforces access control per route.

In production with a real university SSO, only step 2 changes — the frontend redirects to the real Identity Provider, which redirects back with a real token. Steps 3–6 remain identical.

4 Part 4 — The Frontend

The frontend is a **React 18 + Vite** SPA with **React Router v6**, deployed to GitHub Pages. No UI component library — all styling is custom CSS.

Contexts

`AuthContext` — wraps the entire app. Stores the decoded user object (`{ userId, role, fullName }`), the raw token, and provides `login()` / `logout()` methods. `login()` stores the token in `localStorage`; on page refresh it reads the token back and re-validates it.

`LanguageContext` — provides `{ t, language, setLanguage }`. The `t(key)` function looks up a key in the `translations.js` map (supports `en` and `ro`). Used across all pages and navigation to support bilingual display.

Routing (`App.jsx`)

Routes fall into three categories:

Public — accessible to anyone (no authentication required):

- `/` — `PublicPortal` (the landing page for unauthenticated visitors)
- `/login` — `Login`
- `/register` — `Register`
- `/system-diagrams` — `SystemDiagramsPage` (architecture diagram viewer)

Protected — requires any authenticated user. Wrapped in `ProtectedRoute` which redirects to `/login` if no token:

- `/dashboard`, `/documents`, `/privacy`, `/contact`, `/help`, `/shortcuts`, `/docs/:role?`

Role-restricted — inside the protected zone but additionally checked by `RoleRoute`:

- `/my-grades` — STUDENT only
- `/students` — PROFESSOR, SECRETARIAT, ADMIN
- `/disciplines` — ADMIN, SECRETARIAT
- `/grades/list` — PROFESSOR, ADMIN, SECRETARIAT
- `/centralizer` — SECRETARIAT, ADMIN
- `/groups` — SECRETARIAT, ADMIN
- `/audit` — ADMIN only

Page-by-Page Tour

`PublicPortal` — The unauthenticated landing page. Fetches the portal notice and institution name from `/api/public`. Shows the faculty branding and a call to action to log in.

`Login` — Presents a form. On submit it calls the mock SSO then exchanges the token. On success, updates `AuthContext` and navigates to `/dashboard`.

`Register` — Direct registration (non-SSO path). Creates a user account with STUDENT role.

`Dashboard` — The home screen after login. Shows a role-specific stats grid (different cards depending on whether you're an Admin, Secretariat, Professor, or Student) and a recent activity feed pulled from the audit log. Has offline detection — if `navigator.onLine` is false or the network call fails, it shows an offline banner with a retry button.

`Students` — Available to PROFESSOR, SECRETARIAT, ADMIN. Lists all students from `/api/academic/students` with their formation name, enrollment status, and plan count. Supports add, edit, delete, and bulk Excel/CSV import. Professors see a read-only view.

`Disciplines` — Available to ADMIN and SECRETARIAT. Manages curricula and their disciplines. Allows creating curricula for specializations, adding/editing/deleting disciplines, and taking curriculum snapshots.

`GradesList` — Available to PROFESSOR, ADMIN, SECRETARIAT. Allows entering and viewing grades by student and discipline. Professors see their own grades; Secretariat and Admin can see all. Supports bulk import from Excel.

`MyGrades` — STUDENT only. Displays the authenticated student's grades, GPA, ECTS credits earned, and their percentile rank among all students.

`Centralizer` — Available to SECRETARIAT and ADMIN. A powerful filtered view of all grades across the faculty. Filters: specialization, degree level, study year, education form, semester, group, curriculum, discipline, academic year, and free-text search. Results can be exported to PDF (client-side generation via `pdfUtils.js` / `centralizerGenerator.js`), CSV, XLSX, or XML (server-side). The PDF generation uses `jsPDF` or a custom utility that formats the data as a styled table.

`Documents` — Available to all authenticated users (with role-appropriate actions). Students can submit documents (upload files). Professors and Secretariat can review, approve, reject, request modifications, and forward documents to other users. Full-text search across title and content. Supports downloading the original file.

`UserGroups` — Available to SECRETARIAT and ADMIN. Create and manage user groups, add/remove members. Groups are used as targets for mass notification emails.

`AuditLogs` — ADMIN only. Paginated table of all audit log entries showing actor, action type, module, entity type, entity ID, and timestamp. Expanding a row reveals the before/after JSON snapshots. Two recovery actions available per row: **Single Rollback** (undo that one action) and **Point-in-Time Recovery** (restore the entire database to the state at that entry's timestamp).

`Dashboard` **for Admin** specifically shows: total users, total students, active curricula, and audit actions in the last 24 hours.

`Documentation` — a role-aware help page rendered from structured content. Shows different sections based on the `role` route parameter, so each type of user can navigate directly to their relevant docs.

`Help`, `Contact`, `Privacy`, `Shortcuts` — informational pages. `Shortcuts` renders the keyboard shortcut map from the `useKeyboardShortcuts` hook.

Frontend Services (`src/services/api.js`)

An Axios instance pre-configured with `baseURL` (pointing to the backend, configurable via `VITE_API_URL` env var) and a request interceptor that automatically injects the `Authorization: Bearer` header from `localStorage`. Exports named service objects: `auditService`, `adminService`, `academicService`, `documentsService`, `reportsService`, etc., each wrapping the relevant API endpoints.

Utilities

`translations.js` — the full bilingual string map. Every user-visible string has an `en` and `ro` key.

`constants.js` — shared enumerations like document types, status values, and role lists. Kept in sync between frontend and backend.

`pdfUtils.js` / `centralizerGenerator.js` / `transcriptGenerator.js` — client-side PDF/document generation utilities. Used by the Centralizer and the student transcript feature to generate download-ready documents directly in the browser without a server round-trip.

`mockData.js` — seed data used for offline/demo mode and testing.

`useKeyboardShortcuts.js` — a custom React hook that registers global keyboard shortcuts (e.g., `Alt+D` for dashboard, `Alt+S` for students) and cleans them up on unmount.

5 Part 5 — Data Integrity: The Audit & Rollback System

This is the most distinctive engineering choice in the project.

How It Works

Every write operation (INSERT, UPDATE, DELETE) that flows through `auditableInsert`, `auditableUpdate`, or `auditableDelete` in `auditService.js` captures a JSON snapshot of the record's state before and after the operation, within the same database transaction. If the transaction rolls back, the audit entry also rolls back.

This means `AUDIT_LOG_ENTRY` is a complete history of every row in the system.

Single-Record Rollback

When an Admin clicks "Rollback" on an audit log entry:

1. The `POST /api/audit/rollback/:logId` endpoint fetches the log entry.
2. If the original action was `INSERT`: it deletes the record (undoing the creation).
3. If the original action was `UPDATE`: it restores the `before_snapshot_json` columns to the record.
4. If the original action was `DELETE`: it re-inserts the `before_snapshot_json` back into the table.
5. The rollback itself is logged as a new audit entry (action type `ROLLBACK_INSERT`, `ROLLBACK_UPDATE`, or `ROLLBACK_DELETE`), so the rollback is itself auditable and reversible.

The implementation validates `entity_type` against a whitelist (`STUDENT`, `GRADE`, `USER_ACCOUNT`, `DOCUMENT`, `STUDY_FORMATION`, `DISCIPLINE`) to prevent SQL injection.

Point-in-Time Recovery (PITR)

This is the full database rewind feature. When triggered with a `targetTimestamp`:

1. All `AUDIT_LOG_ENTRY` records with `occurred_at > targetTimestamp` are fetched in **descending** order (most recent first).
2. Each entry is reversed: INSERTs become DELETEs, UPDATEs restore their `before_snapshot_json`, DELETEs re-insert their `before_snapshot_json`.
3. The entire operation is wrapped in a single database transaction — either all reversals succeed or none do.
4. A `PITR` log entry is written recording who triggered it, the target timestamp, and how many actions were reversed.

This effectively winds the database back to a consistent snapshot at any point for which audit logs exist.

Backup System

The backup system is a complement to the audit system. While audit logs allow surgical rollback, backups provide a full safety net:

- **Manual backup** — triggered from the Admin UI, calls `createBackup()` which dumps all 16 tables to a timestamped JSON file in `backend/backups/`.
- **Automatic backup** — scheduled via cron expression configured in the Admin UI. The scheduler reads `BACKUP_CONFIG` on startup.
- **Restore from backup** — the Admin can upload a backup JSON file; the system truncates affected tables and re-inserts all rows from the snapshot.
- **Audit archiving** — runs daily at 1 AM. Logs older than 5 years are moved to `AUDIT_LOG_ARCHIVE` to manage active table size while satisfying the institutional 5-year retention policy.

6 Part 6 — The Email Notification System

The system supports three email backends, selected at startup by environment variable:

Graph API (`graphEmailService.js`) — uses `@microsoft/microsoft-graph-client` with Azure identity credentials to send via Microsoft 365. Configured by `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`, `GRAPH_SENDER_EMAIL`.

SMTP (`smtpEmailService.js`) — uses `nodemailer` with standard SMTP credentials. Configured by `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`.

Mock (`mockEmailService.js`) — logs the email payload to the console. Used in development.

The `emailService.js` router checks for the presence of the Azure credentials first; if not found, falls back to SMTP; if not found, falls back to Mock.

The notification flow: Admin or Secretariat selects a group (from `USER_GROUP`) or a study formation, writes a subject and body, and clicks Send. The backend resolves the member emails, calls `emailService.sendBulk()`, and records the send in `OUTLOOK_NOTIFICATION`.

7 Part 7 — The Document Workflow

Documents go through a defined lifecycle:

1. Any authenticated user uploads a document (`POST /api/documents`). Status becomes `DRAFT`.
2. The uploader or a Secretariat/Admin changes the status to `PENDING` to submit it for review.
3. Secretariat or Admin reviews it:
   - `APPROVED` — document is accepted.
   - `REJECTED` — document is declined (with optional revision notes).
   - `MODIFICATION` — document is sent back with revision notes.
4. If `MODIFICATION`, the author can re-upload a new file version (`PUT /:id/reupload`). Status resets to `PENDING`.

5. A document can also be forwarded to a specific user (`PUT /:id/forward`), setting `assigned_to_user_id`.

All status changes go through `auditableUpdate` so the full workflow history is in the audit log.

File storage: uploaded files are stored on disk under `backend/uploads/`. The system accepts PDF, DOCX, XLSX, CSV, XML, TXT, images, PPT, ZIP, RAR, and video files. For PDF, DOCX, XLSX, TXT, and CSV uploads, the system extracts text content and stores it in the `content` column for full-text search.

8 Part 8 — Reports & The Centralizer

The grade centralizer is the main reporting feature. It is a dynamic, filtered view of the intersection of students, disciplines, and grades.

The backend query (`getCentralizerData` in `reports.js`) performs a complex multi-join: it starts from `STUDENT`, joins through `STUDENT_CURRICULUM` → `STUDY_FORMATION` → `CURRICULUM` → `SPECIALIZATION`, then uses a lateral join to get all disciplines either from the curriculum or from existing grades for that student. Grades are outer-joined so students without grades still appear.

Filters available: specialization, degree level, study year, education form, curriculum, discipline, group index, academic year, semester, evaluation type, and a free-text code/name search.

Export formats:

- **CSV** — via PapaParse, flat rows.
- **XLSX** — via the `xlsx` library, formatted workbook.
- **XML** — via `xml2js`, structured document.
- **PDF** — generated client-side via the `centralizerGenerator.js` utility.

The student transcript (`GET /api/reports/transcript/:studentId`) generates a per-student academic record with all grades organized by semester and curriculum.

9 Part 9 — Configuration & System Settings

The Admin has access to a configuration panel (routes under `/api/config`, frontend in the Dashboard admin section) covering:

System Settings — edit the key-value store: faculty name, institution name, dean name, dean secretary, system email, portal notice. These values are consumed by the public portal and embedded in generated documents.

Academic Years — create and manage academic year records. Setting one as `is_active` automatically deactivates all others.

Specializations — manage academic programs. Toggle active/inactive, set degree level.

Backup Schedule — toggle automatic backups on/off and configure the cron expression. Changes take effect immediately (the scheduler updates its live job).

10 Part 10 — How to Run the Project

Prerequisites: Node.js 18+, PostgreSQL 15+.

Backend setup:

1. Navigate to `backend/`.
2. Copy `.env.example` to `.env` and fill in `DB_*` variables, `JWT_SECRET`, `SSO_MOCK_SECRET`, and optionally email credentials.
3. Run `npm install`.
4. Create the database schema by running `src/schema.sql` against your PostgreSQL instance.
5. Optionally run any `seed_*.js` scripts for initial data.
6. Start with `npm run dev` (nodemon) or `npm start`.

Frontend setup:

1. From the project root.
2. Copy `.env.example` to `.env.local` and set `VITE_API_URL` to your backend URL (e.g., `http://localhost:8`
3. Run `npm install`.
4. Start with `npm run dev` (opens on port 5173 by default).

First login: The mock SSO endpoint (`POST /api/mock-sso/login`) accepts `{ email, role, full_name }`. Use role `ADMIN` for full access on first setup. The frontend's login page calls this automatically when you click the SSO login button.

Summary: Architecture in One Paragraph

AFSMS is a layered system. At the bottom is a PostgreSQL schema organized into six functional modules (security, academics, documents, reports, audit/recovery, configuration). The Node.js/Express backend sits on top, exposing a versioned REST API that enforces JWT authentication and role-based access control on every route. All writes are mediated through the audit service, which captures before/after snapshots within the same transaction, enabling single-record rollback and full point-in-time database recovery. The React frontend consumes this API through an Axios service layer, routes users based on their role, and provides specialized pages for each persona — students see their grades, professors manage grade entry, secretariat handles students and documents, and the admin has full visibility including the audit log and system configuration.

End of AFSMS Walkthrough Guide